Information

Problem Statement

ปัจจุบันเวลาหารค่าใช้จ่ายในกลุ่ม เช่น ค่าอาหาร ค่าที่พัก หรือค่าใช้จ่ายร่วมกัน ผู้จ่ายบิลต้องแจ้งเลข PromptPay ของตัวเองและยอดเงินให้ผู้เข้าร่วมแต่ละคนแยกกัน ซึ่งผู้เข้าร่วมจะต้องเปิดแอปธนาคารแล้วกรอกข้อมูลเหล่านั้นด้วยตัวเอง กระบวนการนี้มีโอกาสผิดพลาดสูง ทั้งกรอกยอดผิด โอนให้ผิดคน หรือกระทั่งการลืมโอน

PromptPay QR Split แก้ปัญหานี้โดยสร้างลิงก์เฉพาะบุคคลให้กับผู้เข้าร่วมแต่ละคน ทำให้ผู้เข้าร่วมแต่ละคนสามารถเปิดลิงก์มา เพื่อเจอ QR Code พร้อมยอดเงินที่ถูกต้องของแต่ละคนได้อย่างสะดวก

What is PromptPay QR Split?

PromptPay QR Split เป็นเว็บแอปพลิเคชันสำหรับหารค่าใช้จ่ายในกลุ่ม โดยระบบจะสร้าง QR Code PromptPay เฉพาะบุคคลให้กับผู้เข้าร่วมแต่ละคน แทนที่ผู้จ่ายบิลจะต้องแจ้งเลข PromptPay และยอดเงินให้ทุกคนด้วยตัวเอง เพียงแค่แชร์ลิงก์ให้แต่ละคน เปิดลิงก์มาก็จะเจอ QR Code พร้อมยอดที่ต้องจ่าย ทำให้ผู้เข้าร่วมสามารถโอนได้เลยผ่าน mobile banking ทุกธนาคาร โดย PromptPay QR Split จะประกอบไปด้วย 5 features สำคัญดังนี้

1. **บัญชีผู้ใช้ -** ผู้จ่ายบิลสมัครสมาชิกและตั้งค่าชื่อพร้อมเบอร์ PromptPay ของตัวเองเพียงครั้งเดียว เมื่อสร้างบิลครั้งถัดไประบบจะดึงข้อมูลเหล่านี้มาให้อัตโนมัติ ไม่ต้องกรอกซ้ำทุกครั้ง
2. **การสร้าง และจัดการ Bill -** ผู้จ่ายบิลสร้าง bill โดยกรอกเพียงชื่อบิลและรายชื่อผู้เข้าร่วมพร้อมยอดเงิน โดยผู้จ่ายบิลสามารถเลือกได้ว่าจะหารเท่ากันหมด หรือกำหนดยอดแต่ละคนด้วยตัวเอง
3. **การสร้าง QR Code -** ระบบสร้าง QR Code PromptPay เฉพาะสำหรับแต่ละคน โดยหน้า QR จะแสดงชื่อผู้เข้าร่วม ยอดที่ต้องจ่าย และชื่อพร้อมเบอร์ PromptPay ของผู้จ่ายบิล เพื่อให้ผู้เข้าร่วมตรวจสอบ ก่อนที่จะ scan
4. **การแชร์ลิงก์ -** ผู้จ่ายบิลคัดลอกลิงก์รายคนเพื่อส่งให้แต่ละคน หรือคัดลอกทุกลิงก์พร้อมกันเป็นข้อความสรุปเพื่อวางใน group chat ได้ทีเดียว โดยไม่ต้องส่งทีละคน
5. **Dashboard ติดตามสถานะ -** ผู้จ่ายบิลเห็นภาพรวมของทุก bill ที่สร้างไว้ พร้อมสถานะว่าใครจ่ายแล้ว ใครยังค้างอยู่ และยอดรวมที่ได้รับและยังขาดอยู่ สามารถ mark ว่าจ่ายแล้วได้ด้วยตัวเอง

User Flow

ฝั่งเจ้าของบิล

1. สมัครสมาชิกครั้งแรก โดยหากยังไม่เคยตั้งค่าโปรไฟล์ ระบบจะพาไปกรอกชื่อพร้อมเบอร์ PromptPay ก่อนใช้งานต่อ (ทำครั้งเดียว ใช้ได้ทุก bill ในครั้งถัดไป)
2. สร้าง bill โดยกรอกชื่อบิลและรายชื่อผู้เข้าร่วมพร้อมยอดเงิน
3. คัดลอกลิงก์รายคนหรือทุกลิงก์พร้อมกัน แล้วส่งผ่าน LINE หรือแชทใดก็ได้
4. รอผู้เข้าร่วมโอนเงิน แล้ว mark ว่าจ่ายแล้วบน dashboard

ฝั่งผู้เข้าร่วม

1. เปิดลิงก์ที่ได้รับจากเจ้าของบิล
2. ตรวจสอบชื่อตัวเอง ยอดเงิน และชื่อพร้อมเบอร์ PromptPay ของเจ้าของบิล
3. เปิดแอปธนาคาร scan QR แล้วโอน
4. แจ้งเจ้าของบิลผ่านช่องทางที่ได้รับลิงก์มา เช่น LINE Chat, Instagram Chat, Discord Chat เป็นต้น

Tech Stack

PromptPay QR Split เลือกใช้ Technology ที่เหมาะสมกับขนาด และเป้าหมายของ project โดยเน้นความเรียบง่ายในการ deploy และความสอดคล้องกันของข้อมูล

Next.js (Frontend + Backend)

ใช้ Next.js เป็นทั้ง frontend และ backend ทำให้ deploy ได้ที่เดียวบน Vercel อีกทั้ง Server-side Rendering ยังช่วยให้หน้า QR โหลดเร็วบนมือถือ

promptpay-qr (npm library)

library ที่รองรับมาตรฐาน EMVCo สำหรับ PromptPay โดยเฉพาะ ซึ่งอยู่ใน ecosystem เดียวกับ Next.js จึงใช้งานได้อย่างสะดวก

PostgreSQL ผ่าน Supabase

ข้อมูลมีความสัมพันธ์ชัดเจนระหว่าง users, sessions และ members จึงเหมาะกับ relational database อย่าง PostgreSQL เพราะ มี ACID compliance รวมถึง Supabase ยังมี Authentication built-in ที่รองรับระบบบัญชีผู้ใช้ได้ทันที

Vercel

รองรับ Next.js แบบ native และ deploy อัตโนมัติทุกครั้งที่ push code ขึ้น GitHub ทำให้ได้ live demo URL พร้อมใช้งานตลอด

Constraints & Limitations

1. **ยืนยัน PromptPay ID กับธนาคารไม่ได้**

ระบบตรวจสอบได้เพียงรูปแบบของเบอร์โทรเท่านั้น ไม่สามารถยืนยันได้ว่าเบอร์นั้นมีบัญชี PromptPay จริงหรือไม่ เพราะไม่มี API จากธนาคาร เจ้าของบิลจึงต้องรับผิดชอบความถูกต้องของเบอร์ที่กรอกเอง

2. **ยืนยันการโอนเงินอัตโนมัติไม่ได้**

ระบบไม่สามารถตรวจจับได้ว่าผู้เข้าร่วมโอนเงินจริงหรือยัง เพราะการเชื่อมต่อกับ banking API ต้องใช้ใบอนุญาต Payment Service Provider ซึ่งไม่สามารถขอได้ในฐานะ third-party developer เจ้าของบิลจึงต้อง mark สถานะด้วยตัวเองบน dashboard

3. **รองรับเฉพาะเบอร์โทรเป็น PromptPay ID**

ผู้ที่ผูก PromptPay ด้วยเลขบัตรประชาชนเพียงอย่างเดียวจะไม่สามารถใช้งานแอปนี้ในฐานะเจ้าของบิลได้ เพราะระบบรับเฉพาะเบอร์โทร 10 หลัก

4. **รองรับเฉพาะสกุลเงินบาท**

ระบบออกแบบมาสำหรับ PromptPay ของไทยโดยเฉพาะ ไม่รองรับสกุลอื่นๆ

5. **ไม่มีประวัติย้อนหลังข้าม session**

ระบบไม่ได้ track ว่าผู้เข้าร่วมคนใดเคยจ่ายช้าหรือค้างชำระในบิลก่อนหน้า เพราะไม่มีการเชื่อมโยงข้อมูลระหว่าง bill

6. **QR ที่ถูกบันทึกไว้ยังโอนได้แม้บิลถูกลบ / ถูกปิด**

PromptPay QR เป็น static QR ที่เข้ารหัสเบอร์ปลายทางกับยอดเงินไว้ในตัวภาพ การสแกนโอนเงินทำงานตรงระหว่างแอปธนาคารกับเครือข่าย PromptPay ไม่ผ่านระบบของเรา ดังนั้นถ้าผู้เข้าร่วมบันทึกรูป QR ไว้ก่อนที่เจ้าของบิลจะลบบิลหรือถูก mark ว่าปิดแล้ว รูปนั้นจะยังใช้โอนเงินได้ตามปกติ ระบบทำได้เพียงบล็อกการเข้าถึงหน้า QR ใหม่เท่านั้น



Architectural Decision Records

Architectural Decision Records

ADR-001: ใช้ Next.js API Routes แทนการแยกอีก repo

ระบบต้องการ backend สำหรับจัดการ session, QR generation และสถานะการชำระเงิน แทนที่จะแยกเป็น backend อีก repo หนึ่ง จึงเลือกใช้ Next.js API Routes เพื่อลดความซับซ้อนในการ deploy

ADR-002: ใช้ Relational Database อย่าง PostgreSQL แทน NoSQL

ข้อมูลของระบบมีโครงสร้างความสัมพันธ์ที่ชัดเจน คือ user มีหลาย session และ session มีหลาย member ทำให้ Relational database จึงเหมาะกว่า NoSQL เพราะมี ACID compliance ที่การันตีความถูกต้องของข้อมูล

ADR-003: รับเฉพาะเบอร์โทรเป็น PromptPay ID ไม่รับเลขบัตรประชาชน

แม้ PromptPay จะรองรับทั้งเบอร์โทรและเลขบัตรประชาชน แต่เลขบัตรประชาชนเป็นข้อมูลส่วนบุคคลที่ละเอียดอ่อน และจะถูกแสดงบนหน้า QR ให้ผู้เข้าร่วมทุกคนเห็น เบอร์โทรจึงเป็นตัวเลือกที่ให้ความเป็นส่วนตัวสูงกว่าโดยที่ยังใช้งานได้เทียบเท่ากัน

ADR-004: ใช้ Monolith แทน Microservices

การแยกระบบออกเป็น microservices จะเพิ่ม complexity ในการ deploy โดยไม่จำเป็น ทำให้การแยก layer ชัดเจนภายใน monolith เดียว (controller / service / repository) จึงให้ประโยชน์ด้านการจัดระเบียบ code ได้เทียบเท่ากัน แต่เรียบง่ายกว่า รวมถึงยังไม่มีความจำเป็นที่จะต้อง scale service ใดเป็นพิเศษ

ADR-005: ใช้ Manual Confirm แทนการยืนยันการชำระเงินอัตโนมัติ

การยืนยันการชำระเงินแบบ real-time ต้องการใช้ใบอนุญาต Payment Service Provider ซึ่งไม่สามารถขอได้ในฐานะ third-party developer ทำให้ Manual confirm จากเจ้าของบิล จึงเป็นทางเลือกทำให้ระบบอยู่ในขอบเขตของกฎหมายโดยไม่ต้องมีใบอนุญาตจากธนาคาร ด้วยเหตุผลเดียวกันนี้ QR ที่แสดงให้ผู้เข้าร่วมสแกนจึงถูก generate ขึ้นเอง ฝั่งระบบตามมาตรฐาน EMVCo ผ่าน library แทนการเรียกใช้ QR ของธนาคาร

ADR-006: Deploy บน Vercel

Vercel รองรับ Next.js แบบ native และ deploy อัตโนมัติทุกครั้งที่ push code ทำให้มี live demo URL ที่เข้าถึงได้ทันที

ADR-007: ใช้ Soft Delete แทน Hard Delete สำหรับ Bill

เมื่อเจ้าของบิลลบ bill ระบบจะ set ค่า deleted_at แทนที่จะลบข้อมูลออกจาก database จริงๆ เพื่อให้ผู้เข้าร่วมที่เปิดลิงก์ QR อยู่เห็นข้อความ "บิลนี้ถูกลบแล้ว" แทนที่จะเจอ error ทั้งนี้ deleted_at เป็นคนละสถานะกับ closed_at (ADR-013) เพราะบิลที่ถูกลบ (deleted_at) คือบิลที่เจ้าของไม่ต้องการใช้งานอีกต่อไป ส่วนบิลที่ถูกปิด (closed_at) คือบิลที่ยังใช้งาน/อ้างอิงได้ปกติ เพียงแต่ล็อคไม่ให้แก้ไขสถานะการจ่ายเงินแล้ว

ADR-008: เพิ่มระบบ Login ด้วย Supabase Auth

เดิม PQS ออกแบบมาให้ stateless ไม่มี login แต่ภายหลังเพิ่ม login เข้ามาเพื่อให้เจ้าของบิลตั้งค่าชื่อและเบอร์ PromptPay ครั้งเดียวแล้วใช้ได้ทุก bill โดยเลือก Supabase Auth เพราะมี built-in อยู่แล้วใน Supabase ที่ใช้เป็น database อยู่แล้ว ไม่ต้องเพิ่ม dependency ใหม่ แต่จะใช้ Google OAuth ใน Google Cloud Console แทนที่จะใช้ default OAuth client ที่ Supabase เตรียมไว้ให้ เพื่อให้หน้า consent screen ตอน login แสดงเป็น domain/branding ของ PQS เอง ไม่ใช่ domain กลางของ Supabase สร้างความน่าเชื่อถือให้ user เห็นว่ากำลัง login เข้าระบบของ PQS จริงๆ

ADR-009: แยกการ Login กับการกรอกโปรไฟล์ออกจากกัน ไม่ใช้ Database Trigger

ตอนที่ user login สำเร็จ ระบบของ Supabase จะรู้จัก user ทันที แต่ users table ของเรายังไม่มีข้อมูล เพราะ display_name และ promptpay_number ต้องให้ user กรอกเอง ทำให้มีช่วงเวลาที่ login แล้วแต่ยังไม่มีข้อมูลโปรไฟล์ ทางเลือกที่คิดไว้มี 2 อย่าง คือ ทางแรก จะใช้ database trigger สร้าง row ใน users table ให้อัตโนมัติทันทีตอน login (แต่ต้องยอม display_name และ promptpay_number เป็นค่าว่างได้ก่อน) อีกทางคือให้แอปสร้าง row เองตอน user กรอกหน้าตั้งค่าโปรไฟล์เสร็จ เมื่อพิจารณาแล้วจึงเลือกใช้ทางที่สอง เพราะจะทำให้ทุก row ใน users table มีข้อมูลครบสมบูรณ์เสมอ ผลคือแอปต้องเช็คเองว่า user คนนี้มีโปรไฟล์อยู่แล้วหรือยัง ถ้ายังไม่มีให้พาไปหน้ากรอกโปรไฟล์ก่อน ใช้วิธีเช็คแบบนี้ได้เหมือนกันไม่ว่า user จะ login ด้วยวิธีไหนก็ตาม

ADR-010: ใช้ JWT Access Token คู่กับ Refresh Token

ระบบต้องการ session ที่สมดุลระหว่างความปลอดภัยและความสะดวกในการใช้งาน แนวทางแรก คือใช้ JWT access token อย่างเดียวโดยไม่มี refresh token ซึ่งทำให้ user ต้อง login ใหม่ทุกครั้งที่ token หมดอายุ เทียบกับใช้ JWT access token คู่กับ refresh token ที่ช่วยให้ Supabase JS client auto refresh ได้เงียบๆ โดย user ไม่รู้สึก จึงเลือกใช้แนวทางที่สอง โดยตั้ง access token อายุ 1 ชั่วโมง (Default ของ

Supabase) ส่วน refresh token ของ Supabase นั้น เพราะ Supabase กำหนดให้ refresh token ไม่มีวันหมดอายุตายตัว และการปล่อยให้ user login ค้างไว้ตลอดไปไม่ปลอดภัย จึงเพิ่ม soft timeout ของแอปเองให้ user ต้อง login ใหม่ทุก 7 วันนับจากการ login ครั้งล่าสุด โดยจะมี popup แจ้ง "session หมดอายุ" และ redirect ไปหน้า login เมื่อครบกำหนด แต่การเช็คนี้ทำงานเฉพาะตอนมี request เกิดขึ้นเท่านั้น จึงเพิ่มการเรียก API ที่ Response ขนาดเล็กทุกๆ 30 นาที เพื่อให้ detect ได้แม้ user เปิดแอปทิ้งไว้เฉยๆ ไม่มี action ใดๆ เลย

ADR-011: ใช้ Fetch API แทน Axios

ระบบต้องใช้ API จึงมีแนวทางที่พิจารณาคือใช้ Axios กับการใช้ Fetch API ที่ built-in อยู่ใน browser แล้ว เลือกใช้ Fetch API เพราะ Axios เหมาะกับ project ที่มี API เส้นเยอะ หรือต้องการ interceptor สำหรับแนบ token หรือ handle error อัตโนมัติ หรือต้องเรียก REST API ภายนอกที่ไม่ใช่ของตัวเอง ซึ่ง PQS ไม่ได้มีเงื่อนไขเหล่านี้ เพราะ มีเส้น API ไม่มาก อีกทั้ง Supabase client จัดการ auth token ให้อยู่แล้ว และ API ทั้งหมดเป็นของตัวเอง

ADR-012: หน้า Pay ของ Anon ใช้ SECURITY DEFINER RPC

เดิม policy ของ anon บนตาราง bills และ members ใช้ USING (TRUE) ซึ่งแม้ตั้งใจให้ query ได้แค่ row ตาม id ที่อยู่ใน URL แต่ในทางเทคนิค policy แบบนี้เปิดให้ anon SELECT ได้ทั้งตาราง ไม่ได้ถูกจำกัดไว้แค่ id เดียว ทำให้ใครก็ตามสามารถ enumerate ดึงชื่อ ยอดเงิน ชื่อบิล และเบอร์ PromptPay ของทุกบิลในระบบออกมาได้ ไม่ใช่แค่บิลที่ตัวเองมีลิงก์ ถือเป็นความเสี่ยงด้าน privacy ที่ขัดกับเจตนาเดิมของหน้า pay ที่ควรเข้าถึงได้เฉพาะผ่านลิงก์ที่เจ้าของบิลแชร์มาเท่านั้น จึงแก้โดยลบ policy แบบเปิดกว้างออก แล้วสร้าง function get_member_payment_info(member_id) แบบ SECURITY DEFINER ที่รับ member id เพียงตัวเดียว และคืนค่าเฉพาะ field ที่จำเป็นต่อการแสดงหน้า pay ของ row นั้น row เดียวเท่านั้น พร้อม REVOKE สิทธิ์ EXECUTE จาก PUBLIC แล้ว GRANT เฉพาะ anon และ authenticated วิธีนี้ทำให้ anon ไม่มีสิทธิ์ SELECT ตรงบนตารางอีกต่อไป เข้าถึงข้อมูลได้เฉพาะผ่าน function ที่ควบคุม scope ไว้ชัดเจน

ADR-013: แยกสถานะ "ปิดบิล" (closed_at) ออกจาก Soft Delete (deleted_at)

ระบบต้องการให้เจ้าของบิลสามารถ "ปิด" บิลที่เก็บเงินครบแล้วได้ เพื่อล็อคไม่ให้สถานะการจ่ายเงิน (is_paid) ของ member ถูกแก้ไขต่อโดยไม่ตั้งใจ แต่บิลและข้อมูลผู้เข้าร่วมยังต้องแสดงผลได้ตามปกติสำหรับใครก็ตามที่เปิดลิงก์ดูอยู่ หากใช้ deleted_at ตัว

เดียวสำหรับทั้งสองความหมายจะทำให้ตีความสถานะของบิลกำกวม เพราะ "ลบ" กับ "ปิดรับชำระ" มีผลลัพธ์ที่ต่างกัน จึงเพิ่ม column closed_at แยกต่างหาก โดย API จะปฏิเสธการแก้ is_paid ของ member ที่อยู่ในบิลที่มี closed_at แล้ว และ validate ค่าที่ส่งมาตอนปิด/เปิดบิล ทำให้ทั้งสองสถานะเป็นอิสระต่อกัน



Schema

Schema

“user หนึ่งคนสร้างได้หลาย bill และ bill หนึ่งมีได้หลาย member”

Users - เก็บข้อมูลของเจ้าของบิลที่สมัครสมาชิก

Column	Type	ความหมาย
id	UUID (PK)	รหัสผู้ใช้ที่ไม่ซ้ำกัน
display_name	VARCHAR(100)	ชื่อที่แสดงบนหน้า QR ให้ผู้เข้าร่วมเห็นว่าโอนให้ใคร
promptpay_number	VARCHAR(10)	เบอร์โทรที่ผูกกับ PromptPay สำหรับรับเงิน
created_at	TIMESTAMP	วันเวลาที่สมัครสมาชิก

Bills - เก็บข้อมูลของเจ้าของบิลที่สมัครสมาชิก

Column	Type	ความหมาย
id	UUID (PK)	รหัสบิลที่ไม่ซ้ำกัน ใช้สร้าง URL ของ dashboard
user_id	UUID (FK)	REF(users.id)
bill_name	VARCHAR(100)	ชื่อบิล
owner_name	VARCHAR(100)	ชื่อเจ้าของบิล จะดึงมาจาก users.display_name ในตอนสร้าง
promptpay_number	VARCHAR(10)	เบอร์ PromptPay จะดึงมาจาก users.promptpay_number ในตอนสร้าง
created_at	TIMESTAMP	วันเวลาที่สร้างบิล
deleted_at	TIMESTAMP (NULLABLE)	วันเวลาที่ลบบิล ถ้าเป็น null

		แปลว่าบิลยังใช้งานอยู่
closed_at	TIMESTAMP (NULLABLE)	วันเวลาที่กำหนดว่าบิลนี้ปิดการใช้งานแล้ว ถ้าเป็น null แปลว่าบิลนี้ยังสามารถดู qr แล้วชำระเงินได้อยู่

Members - เก็บข้อมูลผู้เข้าร่วมแต่ละคนใน bill

Column	Type	ความหมาย
id	UUID (PK)	รหัส member ที่ไม่ซ้ำกัน ใช้สร้าง URL หน้า QR ของแต่ละคน
bill_id	UUID (FK)	REF(bills.id)
member_name	VARCHAR(100)	ชื่อผู้เข้าร่วม กรอกโดยเจ้าของบิล
amount	DECIMAL(10,2)	ยอดเงินที่ต้องจ่าย เก็บทศนิยม 2 ตำแหน่ง เช่น 300.50
is_paid	BOOLEAN	สถานะการจ่ายเงิน อัปเดตโดยเจ้าของบิลด้วยตัวเอง
created_at	TIMESTAMP	วันเวลาที่สร้าง bill ซึ่งเป็นเวลาเดียวกับที่ member ถูกสร้างขึ้น